Cross-Lingual Textual Entailment for Hindi: Adapting Lexical Matching to XLM-RoBERTa Fine-Tuning

Mohammed Ufraan
Department of Computer Science
[University / college name]
naarfu@gmail.com

***Abstract*—Recognizing textual entailment (RTE) asks whether the meaning of a hypothesis can be inferred from a premise. Khader et al. (2016) addressed this problem for Arabic with stemmed word overlap, synonym expansion, and bigram thresholds, reporting 61% overall precision and recall on a 100-pair sample of ArbTEDS. We keep their scientific question—entailment outside English, in a morphologically rich language—but change the language, data, algorithm, and training method. Specifically, we fine-tune XLM-RoBERTa-base on the Hindi split of XNLI for three-way natural language inference (entailment, neutral, contradiction). Evaluation uses the standard 5,010-pair XNLI-Hindi test set rather than a 100-pair subsample. On Apple Silicon we document a training-stack failure (fused AdamW and SDPA) that collapsed the classifier to chance (33.3% accuracy, loss pinned at $\ln 3$), and we replace it with eager attention and unfused AdamW. [After the full run, replace with: the model reaches XX.X% test accuracy and YY.Y% macro-F1.] The assignment required at least two modifications of an existing paper; this work implements five (language, dataset, algorithm, methodology, and a full code release).**

***Index Terms*—textual entailment, natural language inference, Hindi, XLM-RoBERTa, XNLI, transfer learning**

I. Draft checklist (delete before submission)

Red **[markers]** are filled from `output/val_metrics.json`, `results/metrics.json`, and `output/training.log`. Citations are author–year and jump to the reference list.

Placeholder	Source
Test acc. / P / R / F1	`results/metrics.json`
Val acc. / P / R / F1	`output/val_metrics.json`
Train time, samples/s	`output/training.log`
Confusion matrix	`results/confusion_matrix.png`
Author affiliation	fill university name by hand

II. Introduction

Textual entailment, also called natural language inference (NLI), is the problem of deciding whether a hypothesis H is true given a text or premise T (Dagan et al., 2006; Bowman et al., 2015). A system that cannot tell whether one sentence licenses another cannot reliably summarize, answer questions, or filter retrieved passages: those applications all reduce, at some step, to "does this sentence follow from that one?"

The PASCAL Recognising Textual Entailment challenges (Dagan et al., 2006) framed the modern English evaluation as a two-way decision (*entails* / *does not entail*). Later English corpora, especially SNLI (Bowman et al., 2015) and MultiNLI (Williams et al., 2018), added a third *neutral* label for cases where T neither supports nor contradicts H. Most published neural NLI work still concentrates on English. Work on Arabic, Hindi, and other morphologically rich languages remains thinner, even though those languages are exactly where surface-form matching is least reliable.

A. The paper we adapt

Khader et al. (2016) studied RTE for Arabic. Arabic has optional diacritics (so a single undiacritized token can have several readings), flexible word order (VSO, SVO, and others), and a large synonym inventory. Their system is a pipeline of preprocessing (named-entity marks, POS tags from MSTParser and MADA, ISRI stemming), stop-word removal, unigram overlap with synonym credit, and hypothesis-normalized bigram overlap. Each overlap score is compared to a threshold chosen on a development sample. The decision is binary: Entails or NotEntails. On 100 randomly selected pairs from ArbTEDS (Alabbas, 2013) they report 68% precision / 64% recall for Entails, 58% / 60% for NotEntails, and 61% overall precision and recall (their Table 6; 61 of 100 pairs judged correctly in their Table 5).

That design has four structural limits. First, thresholds and bag-of-word overlap do not model long-range context or compositional negation. Second, the authors themselves note, following AL-Khawaldeh (2015), that treating negation as a stop word can reverse polarity. Third, evaluation is a 100-pair subsample of a 618-pair corpus, so confidence intervals are wide. Fourth, the feature stack (ISRI stemmer, Arabic WordNet-style synonyms, CoNLL trees) does not transfer to Hindi.

B. What we change

We keep the same research question—can we recognize entailment in a morphologically rich language other than English—and change the implementation along the axes the course assignment allows (Table I). Hindi, like Arabic, has rich morphology and relatively free word order, and it is

TABLE I
COURSE OPTIONS AND HOW THIS PROJECT MEETS THEM. AT LEAST TWO WERE REQUIRED.

#	Option	This work
1	Different language	Arabic → Hindi
2	Improved results	61% P/R → target mid-70s acc. on XNLI-hi
3	Changed algorithm	Overlap rules → Transformer
4	Changed methodology	Thresholds → gradient fine-tuning
5	Full implementation	`src/train.py`, `eval_nli.py`, `inference.py`
6	Different dataset	ArbTEDS → XNLI Hindi

written in Devanagari rather than Arabic script, so a successful transfer cannot be a copy-paste of stemmers and lexicons. We therefore replace the lexical pipeline with a pretrained multilingual Transformer (Conneau et al., 2020) fine-tuned on XNLI Hindi (Conneau et al., 2018).

Option 2 must be stated carefully. Khader et al. (2016) report two-way precision/recall on 100 Arabic pairs. We report three-way accuracy on 5,010 Hindi pairs. If our test accuracy is well above 61%, the assignment criterion "improved results" is met, but it is *not* a matched leaderboard. Published XLM-R-base scores on XNLI-Hindi sit in the mid-70% range (Conneau et al., 2020); that is the honest target. An 85%+ claim would require a larger model or extra data and is not assumed here.

C. Contributions

This report makes four concrete contributions:

1) A complete, runnable fine-tuning recipe for XLM-RoBERTa-base on XNLI Hindi (batch 16, length 128, learning rate 2×10^{-5}, 10% warmup, three epochs on the full train split).
2) A documented MPS failure mode: default fused AdamW in Transformers v5 plus SDPA produced a collapsed $\langle s \rangle$ representation, training loss $\approx \ln 3$, and 33.33% validation accuracy with macro-precision 0.111. The fix is eager attention and unfused AdamW.
3) A comparison, with caveats, against the Arabic lexical baseline of Khader et al. (2016).
4) An IEEE-format write-up and public scripts so the experiment can be reproduced on Apple Silicon without a CUDA GPU.

Section III reviews RTE, Arabic TE, and multilingual encoders. Section IV sketches why Hindi is a fair analogue of Arabic for this assignment. Section V reconstructs the original lexical method. Section VI describes XNLI Hindi. Section VII specifies the model and optimizer. Section VIII gives the experimental protocol. Section IX holds results (placeholders until the run finishes). Section X discusses limitations and Section XII concludes.

III. RELATED WORK

A. English RTE and NLI

Early RTE systems mixed WordNet, logical inference, n-gram overlap, and syntactic matching (Dagan et al., 2006). Du et al. (2010) combine unigram overlap with WordNet and give extra weight to POS and named entities—a recipe close to Khader et al. (2016). Pakray et al. (2011) add an SVM over 23 lexical features plus a rule-based syntactic tag, scoring the mid-50s on RTE-4. These systems show that overlap is a surprisingly strong baseline when T and H share many content words, and a weak one when entailment depends on a single negation, a paraphrase, or a world-knowledge fact.

Large annotated corpora changed the method. SNLI (Bowman et al., 2015) provided hundreds of thousands of English pairs; MultiNLI (Williams et al., 2018) added genre diversity. Neural classifiers trained on those corpora quickly overtook feature pipelines. BERT (Devlin et al., 2019) made the standard recipe: concatenate T and H, encode with a Transformer (Vaswani et al., 2017), and classify the first-token state. RoBERTa (Liu et al., 2019) kept the architecture and improved the pretraining recipe (more data, no next-sentence prediction, dynamic masking).

B. Arabic textual entailment

Arabic TE is small as a literature. Alabbas (2011) first laid out why off-the-shelf English RTE tools fail on Arabic (diacritics, flexible order, parser noise) and experimented with tree-edit distance on dependency trees. Alabbas et al. (2013) extend TED with subtree operations. ArbTEDS (Alabbas, 2013) collects 618 news headline–article pairs, triple-annotated, covering sport, politics, business, and general news. AL-Khawaldeh (2015) show that dropping Arabic negation as a stop word flips Entails/NotEntails. Khader et al. (2016) sit in this line: they keep the ArbTEDS setting and a lexical/semantic pipeline rather than TED, and they evaluate with RTE-2 style precision and recall.

C. Cross-lingual NLI and XLM-R

XNLI (Conneau et al., 2018) translates MultiNLI development and test data into 14 other languages (15 including English), among them Hindi, and provides a training corpus in each language. Professional translators produce the evaluation sets, so the Hindi test set is not crowd-sourced paraphrase of Hindi news; it is translated English NLI. That is a limitation we return to in Section X.

Cross-lingual language model pretraining (Lample et al., 2019) showed that a shared subword vocabulary and a masked-language-model objective transfer to XNLI. XLM-RoBERTa (Conneau et al., 2020) scales that idea: RoBERTa-style training on 2.5 TB of CommonCrawl in 100 languages, including Hindi, with a 250k SentencePiece vocabulary (Kudo et al., 2018). The base model we use has 12 layers, hidden size 768, and about 278M parameters once a three-way classification head is attached. Conneau et al. (2020) report XLM-R-base in the mid-70s accuracy on XNLI Hindi; XLM-R-large is higher at much larger cost. We implement fine-tuning with Hugging Face Transformers (Wolf et al., 2020).

IV. WHY HINDI

The assignment asks for a *different language*, not a random one. Hindi is a reasonable counterpart to Arabic for RTE for four reasons.

a) Morphology.: Hindi is Indo-Aryan, not Semitic, but it is still morphologically rich: verbs inflect for tense, aspect, mood, and honorifics; nouns inflect for number, case, and gender. A surface unigram match therefore misses inflected forms of the same lemma in the same way Arabic overlap misses undiacritized roots. Subword SentencePiece units (Kudo et al., 2018) are a coarse substitute for stemming: they split frequent suffixes rather than recovering a linguistic root, but they do not require an ISRI-style analyser.

b) Word order.: Hindi is canonically SOV and allows scrambling of arguments. Entailment can survive a permutation of the same case-marked noun phrases, which bag-of-word overlap captures and which bigrams can miss. A Transformer attends across the whole 128-token window, so order is a feature rather than a hard template.

c) Script and tokenization.: Devanagari is an abugida, not an alphabet. Whitespace tokenization already yields akshara sequences; XLM-R's 250k vocabulary includes Hindi pieces learned from CommonCrawl (Conneau et al., 2020). We do not transliterate, unlike ArbTEDS's Buckwalter form (Alabbas, 2013).

d) Resource profile.: Hindi has web-scale unlabeled text (hence XLM-R) and a public NLI benchmark (XNLI), but it does not have a widely used 618-pair news TE corpus analogous to ArbTEDS. Using XNLI is therefore both a dataset change and a necessity: there is no drop-in Hindi ArbTEDS. Code-mixing (Hindi–English) appears in real user text and only sparsely in XNLI translations; we do not claim robustness to social-media Hinglish.

Taken together, switching Arabic to Hindi is not a cosmetic find-and-replace. It forces a different script, a different morphology, and a different public dataset, which is why options 1 and 6 of the assignment are satisfied simultaneously.

V. The Original Lexical Method

Reconstructing Khader et al. (2016) makes the contrast with fine-tuning precise.

A. Preprocessing

Pairs are reversed from Buckwalter transliteration into Arabic script, then tagged and stemmed. Named entities are marked in the released dataset. POS tags combine MSTParser and MADA (the authors cite 81% tagging accuracy on the Penn Arabic Treebank). Stems come from the ISRI stemmer. Stop words are then removed, with the acknowledged risk that negation particles are lost (AL-Khawaldeh, 2015).

B. Unigram overlap and synonyms

Let T and H be bags of stems after stop-word removal. Word overlap is the size of the intersection, with synonyms of a token in T counting as a match to H (the paper's example is *ibn* / *najl*, both "son"). Entailment is predicted only if the overlap exceeds a threshold τ_u chosen by experiment. This is a linear decision in a one-dimensional feature:

$$\mathrm{overlap}(T,H) = \big|\{\, w \in H : w \in T \;\vee\; \mathrm{syn}(w) \cap T \neq \emptyset \,\}\big|. \tag{1}$$

TABLE II
XNLI Hindi split sizes.

Split	Pairs	Role
Train	392,702	fine-tuning
Validation	2,490	checkpoint selection
Test	5,010	held-out report

No learning is applied to τ_u beyond grid search on the 100-pair sample.

C. Bigram match

Following Du et al. (2010), they compute

$$\mathrm{BigramMatch}(T,H) = \frac{\#\{\text{bigrams of } H \text{ that occur in } T\}}{\#\{\text{bigrams of } H\}}. \tag{2}$$

A second threshold τ_b is applied. The final label is Entails if both tests pass, otherwise NotEntails. The architecture (their Figure 1) is strictly feed-forward: preprocess, lexical/semantic matching, binary decision. There is no training set in the neural sense; the 100 pairs are both the development sample for thresholds and the reported test set.

D. Why this fails to port to Hindi

A Hindi replica would need a Hindi stemmer, a Hindi synonym lexicon, Hindi stop lists, and a new pair of thresholds. Even then, three-way XNLI labels have no equivalent of τ_u: "neutral" is not "overlap in between." Neural fine-tuning sidesteps that feature engineering. The encoder sees subword IDs, including negation particles, and the classification head is trained with cross-entropy on 392k labeled pairs.

The authors close by suggesting POS-weighted overlap and extra credit for named entities (Du et al., 2010) as future work. They do not propose a neural model. Our change of algorithm (option 3) is therefore a break from their stated roadmap, not an implementation of a paragraph they already wrote.

VI. Dataset

A. XNLI Hindi

We use the Hugging Face build `facebook/xnli`, configuration `hi`. Each row is a Unicode Hindi premise, a Unicode Hindi hypothesis, and a `ClassLabel` in $\{0,1,2\}$: entailment, neutral, contradiction. Table II lists split sizes. Training pairs come from the XNLI training dump; validation and test are the professionally translated MultiNLI matched sets (Conneau et al., 2018). Genres include fiction, government, slate, telephone, travel, letters, the 9/11 report, and others, matching MultiNLI (Williams et al., 2018).

Labels are roughly balanced. On a 5% prefix of the train split (19,635 pairs, the sanity run) we counted 6,648 entailment, 5,836 neutral, and 7,151 contradiction. A majority-class predictor on a balanced three-way test set scores about 33.3% accuracy. Always predicting one class yields macro-precision $1/9 \approx 0.111$ and macro-recall $1/3 \approx 0.333$. We treat that metric fingerprint as a *failed training run*, not as a model result.

TABLE III
ILLUSTRATIVE HINDI PAIRS (DEMO SET, NOT XNLI).

Premise	Hypothesis	Label
The cat is sleeping indoors.	An animal is sleeping in the house.	E
It is raining.	Vehicles are moving on the road.	N
The weather is very good today.	It is raining heavily today.	C

English glosses of the Devanagari strings in `src/inference.py`. E/N/C = entailment/neutral/contradiction.

B. Contrast with ArbTEDS

ArbTEDS (Alabbas, 2013) has 618 pairs; Khader et al. (2016) test on 100 of them (42 Entails, 58 NotEntails). Hypotheses are news headlines and texts are article sentences, collected with a Google API and triple-annotated. The corpus is binary and two orders of magnitude smaller than XNLI-Hindi train. Direct numeric comparison is therefore only a rubric check ("did we beat 61% on our task?"), not a claim that XLM-R would score 75% on ArbTEDS or that overlap would score 61% on XNLI-hi.

C. Input format and examples

Table III shows hand-written Hindi pairs used in `src/inference.py` to smoke-test a saved model. They are not from XNLI and are not used in the official score. Tokenization uses the XLM-R SentencePiece model. A pair is serialized as

$$\langle s\rangle \textit{ premise } \langle /s\rangle\langle /s\rangle \textit{ hypothesis } \langle /s\rangle$$

and truncated or padded to 128 tokens. Length 128 is the XNLI fine-tuning default in much of the literature; longer premises are truncated from the right. On a 64-example probe, non-pad lengths ranged from 16 to 128 (mean ≈ 69), so 128 covers most Hindi XNLI pairs but not all.

VII. METHOD

A. Encoder

Let $x = (x_1, \ldots, x_n)$ be the tokenized pair with $n \le 128$. XLM-RoBERTa (Conneau et al., 2020) is a 12-block Transformer encoder (Vaswani et al., 2017). In each block, queries, keys, and values Q, K, V for a head are linear maps of the residual stream, and

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V \tag{3}$$

with $d_h = 64$ (12 heads $\times$ 64 $=$ 768). A position-wise MLP then expands to 3,072 units and back. Residual connections and layer normalization follow the RoBERTa (Liu et al., 2019) layout. There is no token-type embedding (XLM-R's type vocabulary size is 1); two consecutive separator tokens mark the premise/hypothesis boundary. We load `xlm-roberta-base` and replace the masked-language-model head with a sequence-classification head. All 278M parameters are trainable; we do not freeze the encoder. Dropout 0.1 remains at the values shipped in the pretrained config.

TABLE IV
FINE-TUNING HYPERPARAMETERS (PAPER RUN).

Setting	Value
Model	`xlm-roberta-base`
Parameters (with head)	278,045,955
Labels	3 (E / N / C)
Max length	128
Train batch size	16
Eval batch size	32
Epochs	3 (full data)
Learning rate	2×10^{-5}
Warmup	10% of steps
Optimizer	AdamW, unfused
Weight decay	0.01
Attention	eager (not SDPA)
Precision	FP32
Device	Apple M5, MPS
Seed	42

B. Classification head

Let $h_{\langle s\rangle} \in \mathbb{R}^{768}$ be the final-layer hidden state at the first token. The RoBERTa classification head is

$$z = \tanh(W_d h_{\langle s\rangle} + b_d), \tag{4}$$

$$\ell = W_o z + b_o \in \mathbb{R}^3, \tag{5}$$

with $W_d \in \mathbb{R}^{768\times768}$ and $W_o \in \mathbb{R}^{3\times768}$ randomly initialized (the LOAD REPORT's "MISSING" rows). Dropout is applied before both linear maps according to the model config. Predicted class is $\arg\max_k \ell_k$. Training minimizes token-level cross-entropy

$$\mathcal{L} = -\log \frac{\exp(\ell_y)}{\sum_{k=0}^{2}\exp(\ell_k)}, \tag{6}$$

where $y \in \{0, 1, 2\}$ is the gold label. A uniform classifier has $\mathbb{E}[\mathcal{L}] = \ln 3 \approx 1.099$, which is the loss we observed when training collapsed.

C. Optimization

We use AdamW (Loshchilov et al., 2019; Kingma et al., 2015) with decoupled weight decay 0.01, learning rate 2×10^{-5}, and a linear schedule with 10% warmup. In Transformers 5.x the field `warmup_steps` accepts a float in $[0, 1)$ as a fraction of total steps; we set `warmup_steps=0.1`. Batch size is 16, sequence length 128, seed 42, full precision (FP32). Checkpoints are written each epoch; the best validation accuracy is restored at the end. Table IV summarizes the paper-run settings.

D. Metrics

We report accuracy and *macro*-averaged precision, recall, and F1. Macro averaging treats the three classes equally, so a one-class predictor cannot hide behind 33% accuracy. Khader et al. (2016) reported per-label and overall precision/recall for a two-way task; we cite their 61% overall P/R as the original baseline and do not convert it into a three-way F1.

E. Apple Silicon training stack

Transformers v5, when PyTorch $\geq$ 2.8, defaults to `adamw_torch_fused`. Fused AdamW on MPS is a beta kernel. Combined with the library default of scaled dot-product attention (SDPA), a 5%/1-epoch run produced:

- training loss stuck at 1.09–1.11 for 1,228 steps;
- healthy-looking gradient norms (0.9–6.9);
- validation accuracy exactly $1/3$, macro-precision 0.1111, macro-recall 0.3333, macro-F1 0.1667;
- $\langle s \rangle$ hidden states with cosine similarity ≈ 1 across different pairs (representation collapse), even though encoder weights had moved only $\sim 10^{-3}$ from initialization.

The XNLI columns, tokenizer pairing, and `label`→`labels` collator were intact (diverse `input_ids`, mixed labels). The failure was the optimizer/attention path, not missing targets. The production code therefore sets `attn_implementation="eager"` and `optim="adamw_torch"`, and does not call `model.to("mps")` before the Trainer (the Trainer places the model). After that change, a 5% run showed train loss falling through 1.05 then below 1.0 in the second half of the epoch—the signature of an actual learning signal.

VIII. Experiments

A. Software

Python 3.14, PyTorch 2.12–2.14, Transformers 5.17, Datasets, and Evaluate. Scripts: `src/train.py` (fine-tune), `src/eval_nli.py` (test metrics and confusion matrix), `src/inference.py` (six Hindi demo pairs). Tokenization writes an explicit `labels` column and drops raw text. Logging uses a tqdm bar every step and loss every 10 steps. Hardware: Apple M5, 25.8 GB RAM, Metal Performance Shaders.

B. Protocol

1) Load `facebook/xnli` / `hi`. Optionally take the first p fraction of train (`--subset`).
2) Tokenize premise–hypothesis pairs to length 128.
3) Fine-tune as in Table IV. Select the epoch with highest validation accuracy.
4) Save tokenizer and weights to `output/model`.
5) Run `src/eval_nli.py` on the 5,010-pair test split; write `results/metrics.json` and `results/confusion_matrix.png`.

C. Runs

Sanity (5%, 1 epoch). 19,635 pairs, 1,227 steps/epoch, about 25 minutes at ∼13 samples/s. Purpose: confirm loss leaves $\ln 3$ and validation accuracy leaves 33.3%. **[Sanity val acc. / F1 from the current timing-run.]**

Paper (100%, 3 epochs). 392,702 pairs. This is the number that goes in Table V. **[Wall-clock and throughput from `output/training.log`.]** At the sanity throughput, three full epochs are on the order of a day; we report whatever the machine actually measured.

TABLE V
MAIN RESULTS ON XNLI HINDI. REPLACE RED CELLS AFTER THE RUN.

Split	Acc.	$\mathrm{P}_{\mathrm{mac}}$	$\mathrm{R}_{\mathrm{mac}}$	$\mathrm{F1}_{\mathrm{mac}}$
Validation	[0.xxx]	[0.xxx]	[0.xxx]	[0.xxx]
Test (5,010)	[0.xxx]	[0.xxx]	[0.xxx]	[0.xxx]

TABLE VI
KHADER ET AL. VS. THIS WORK. NOT A MATCHED-TASK LEADERBOARD.

	Khader et al. (2016)	This work
Language	Arabic	Hindi
Dataset	ArbTEDS, 100 / 618 pairs	XNLI-hi, 5k test
Labels	Entails / NotEntails	E / N / C
Features	stems, synonyms, bigrams	subword IDs
Decision	two thresholds	softmax over 3 logits
Score	61% P/R	**[XX.X% acc.]**
Hardware	CPU, rule-based	Apple M5, MPS

We do not report the fused-AdamW collapse as a model score. It is an implementation note so that other MPS users do not treat 33.3% as "XNLI is too small."

D. Baselines we compare against

- Uniform / collapsed class: 33.3% accuracy.
- Khader et al. (2016): 61% P/R, two-way, 100 Arabic pairs.
- Published XLM-R-base on XNLI-Hindi: mid-70s accuracy (Conneau et al., 2020).

A lexical-overlap baseline on Hindi XNLI is possible but was not required for the assignment; we leave it as future work rather than invent a number.

IX. Results

A. Main scores

Table V is filled after `src/eval_nli.py` completes. Until then every cell is a placeholder.

A successful paper run should be far from 33.3% and in the vicinity of the published XLM-R-base band. Macro-F1 should track accuracy if classes stay balanced in error; a large gap would mean one class is ignored.

a) What would count as success.: If test accuracy is 33.3%, the run failed (collapsed head). If it is in the 50–65% band on a 5%/1-epoch sanity check, the pipeline is learning and the full three-epoch run is worth the wall-clock. If the full run lands in the mid-70s, we have reproduced the expected XLM-R-base level (Conneau et al., 2020) and clearly beaten 61% P/R on a harder label set. If it exceeds 80%, we should sanity-check for test leakage (there should be none: XNLI test is a fixed split). We will not claim 85% unless the JSON file actually contains that number.

B. Comparison with the original paper

Table VI is the assignment comparison. Caption states the mismatch of tasks.

C. Error analysis

After evaluation, include `results/confusion_matrix.png`:

[Figure:** 3×3 **confusion matrix, XNLI-Hindi test.]

XNLI errors are typically (i) neutral vs. entailment when H adds a plausible extra fact, and (ii) contradiction carried by a single negation or antonym. Overlap systems (Khader et al., 2016; Du et al., 2010) miss (ii) if negation is dropped as a stop word. The Transformer keeps those tokens, but it can still fail when the contradiction is pragmatic rather than lexical. **[Two or three real Hindi examples from the test set once predictions.npy exists.]**

D. Training dynamics

On a healthy run, logged loss starts near 1.10 (uniform), stays noisy through warmup ($\sim$10% of steps), then trends down. The 5% run that used eager attention reached the high-0.9s by $\sim$80% of epoch 1. Ten-step averages jitter by $\pm$0.03; that is not collapse. Collapse looks like a flat 1.10 for the entire epoch plus 33.33% validation accuracy. **[Final train loss; val accuracy after each epoch of the paper run.]**

X. Discussion and Limitations

a) Translated test set.: XNLI-Hindi hypotheses are translations of English MultiNLI (Conneau et al., 2018). A model can exploit translationese (more explicit pronouns, more regular word order) that would not appear in naturally authored Hindi news. ArbTEDS, by contrast, is naturally Arabic. That is another reason not to over-interpret a 15-point gap versus Khader et al. (2016).

b) Length 128.: Some XNLI premises are multi-sentence. Truncation can delete the clause that licenses or blocks H. A 256-token run is a cheap ablation we did not schedule on M5.

c) Base vs. large.: XLM-R-large would likely raise accuracy several points (Conneau et al., 2020) at roughly $4\times$ memory and time. For a course deadline, base is the right default.

d) No matched Arabic rerun.: We did not reimplement overlap+bigram on Hindi XNLI, nor XLM-R on ArbTEDS. Either experiment would tighten the comparison; both are future work.

e) Hardware specificity.: The fused-AdamW/SDPA collapse is an MPS observation. CUDA users can keep fused kernels. The scientific result (fine-tuning XLM-R on XNLI-hi) does not depend on Apple hardware; the engineering note does.

f) Ethics.: XNLI contains MultiNLI genres (including the 9/11 report). We use the corpus only as a public benchmark. The demo pairs in `inference.py` are synthetic and contain no personal data.

XI. Reproducibility

From the repository root, after `bash setup.sh` and activating `.venv`:

```
python train.py --epochs 1 --subset 0.05 \
    --output-dir ./timing-run
python train.py --output-dir ./output
python eval_nli.py --model-dir output/model
make paper
```

Seeds, learning rate, and attention implementation are fixed in `src/train.py`. Fill every **[marker]** from the JSON/log files, then delete Section I and the `\TBD` macro.

XII. Conclusion

Khader et al. (2016) showed that lexical overlap plus synonyms can reach 61% P/R on a small Arabic RTE sample. We asked the same question for Hindi and answered it with a multilingual Transformer. Changing language, dataset, algorithm, and training method—and shipping runnable code—satisfies the course requirement of at least two adaptations, and in practice five of the six listed options. The remaining option (improved results) is an empirical claim: **[test accuracy XX.X%, macro-F1 YY.Y%, against 61% P/R in the original system and the mid-70s XLM-R-base XNLI-Hindi range in prior work (Conneau et al., 2020).]**

Future work is straightforward: XLM-R-large, length 256, a Hindi-original test set, and a true overlap baseline on the same pairs so the lexical vs. neural gap is measured in one language.

Acknowledgment

Prepared as a deep learning course assignment adapting Khader et al. (2016).

References

AL-Khawaldeh, F. T. (2015). "A Study of the Effect of Resolving Negation and Sentiment Analysis in Recognizing Text Entailment for Arabic". In: *World of Computer Science and Information Technology Journal* 5.7, pp. 124–128.

Alabbas, M. (2011). "ArbTE: Arabic Textual Entailment". In: *Proceedings of the 2nd Student Research Workshop associated with RANLP*, pp. 48–53.

– (2013). "A Dataset for Arabic Textual Entailment". In: *Proceedings of the Student Research Workshop associated with RANLP*, pp. 7–13.

Alabbas, M. and A. Ramsay (2013). "Natural Language Inference for Arabic Using Extended Tree Edit Distance with Subtrees". In: *Journal of Artificial Intelligence Research* 48, pp. 1–22.

Bowman, S. R. et al. (2015). "A Large Annotated Corpus for Learning Natural Language Inference". In: *Proceedings of EMNLP*, pp. 632–642.

Conneau, A. et al. (2018). "XNLI: Evaluating Cross-lingual Sentence Representations". In: *Proceedings of EMNLP*, pp. 2475–2485.

Conneau, A. et al. (2020). “Unsupervised Cross-lingual Representation Learning at Scale”. In: *Proceedings of ACL*, pp. 8440–8451.

Dagan, I., O. Glickman, and B. Magnini (2006). “The PASCAL Recognising Textual Entailment Challenge”. In: *Machine Learning Challenges*, pp. 177–190.

Devlin, J. et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*, pp. 4171–4186.

Du, Y., C. Yao, and J. Liu (2010). “Recognize Textual Entailment by the Lexical and Semantic Matching”. In: *Proceedings of ICCASM*, pp. V2–500–V2–504.

Khader, M., A. Awajan, and A. Alkouz (2016). “Textual Entailment for Arabic Language based on Lexical and Semantic Matching”. In: *International Journal of Computing & Information Sciences* 12.1, pp. 67–74.

Kingma, D. P. and J. Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations*.

Kudo, T. and J. Richardson (2018). “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”. In: *Proceedings of EMNLP: System Demonstrations*, pp. 66–71.

Lample, G. and A. Conneau (2019). “Cross-lingual Language Model Pretraining”. In: *Advances in Neural Information Processing Systems*.

Liu, Y. et al. (2019). “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. In: *arXiv preprint arXiv:1907.11692*.

Loshchilov, I. and F. Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*.

Pakray, P., S. Bandyopadhyay, and A. Gelbukh (2011). “Textual Entailment using Lexical and Syntactic Similarity”. In: *International Journal of Artificial Intelligence & Applications* 2.1, pp. 43–58.

Vaswani, A. et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30.

Williams, A., N. Nangia, and S. R. Bowman (2018). “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference”. In: *Proceedings of NAACL-HLT*, pp. 1112–1122.

Wolf, T. et al. (2020). “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of EMNLP: System Demonstrations*, pp. 38–45.